1. Two Sum (LeetCode 1)

Pattern

Hash Map (Number → Index)

```
class Solution {
public:
   vector<int> twoSum(vector<int>& nums, int target) {

      unordered_map<int,int> m;

      for(int i=0;i<nums.size();i++)
      {
         int complement = target - nums[i];

         if(m.find(complement) != m.end())
            return {m[complement], i};

         m[nums[i]] = i;
      }

      return {};
   }
};
```

Remember

- Compute complement.
- Check if complement already exists.
- Otherwise store current number.

TC: `O(n)` | **SC:** `O(n)`

2. Remove Duplicates from Sorted Array (26)

Pattern

Two Pointers (Reader-Writer)

```
class Solution {
public:
    int removeDuplicates(vector<int>& nums) {

        if(nums.size()==0)
            return 0;

        int k=1;

        for(int i=1;i<nums.size();i++)
        {
            if(nums[i]!=nums[k-1])
            {
                nums[k]=nums[i];
                k++;
            }
        }

        return k;
    }
};
```

Remember

- Array is sorted.
- Writer stores unique elements.

TC: `O(n)` | **SC:** `O(1)`

3. Remove Element (27)

Pattern

Two Pointers (Reader-Writer)

```
class Solution {
public:
    int removeElement(vector<int>& nums, int val) {

        int k=0;

        for(int i=0;i<nums.size();i++)
        {
            if(nums[i]!=val)
            {
                nums[k]=nums[i];
                k++;
            }
        }

        return k;
    }
};
```

Remember

Keep every element except `val`.

TC: `O(n)` | **SC:** `O(1)`

4. Best Time to Buy and Sell Stock (121)

Pattern

Greedy + Running Minimum

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {

        int minPrice = INT_MAX;
        int maxProfit = 0;

        for(int price : prices)
        {
            minPrice = min(minPrice, price);

            int profit = price - minPrice;

            maxProfit = max(maxProfit, profit);
        }

        return maxProfit;
    }
};
```

Remember

Maintain:

- Minimum buying price
- Maximum profit

TC: `O(n)` | **SC:** `O(1)`

5. Majority Element (169)

Pattern

Boyer-Moore Voting

```
class Solution {
public:
    int majorityElement(vector<int>& nums) {

        int candidate;
        int count=0;

        for(int num : nums)
        {
            if(count==0)
                candidate=num;

            if(num==candidate)
                count++;
            else
                count--;
        }

        return candidate;
    }
};
```

Remember

Different elements cancel each other.

TC: `O(n)` | **SC:** `O(1)`

6. Contains Duplicate (217)

Pattern

Hash Set

```
class Solution {
public:
   bool containsDuplicate(vector<int>& nums) {

       unordered_set<int> s;

       for(int num : nums)
       {
          if(s.find(num)!=s.end())
             return true;

          s.insert(num);
       }

       return false;
   }
};
```

Remember

Ask:

Have I seen this number before?

TC: `O(n)` | **SC:** `O(n)`

7. Valid Anagram (242)

Pattern

Frequency Counting

```
class Solution {
public:
    bool isAnagram(string s, string t) {

        if(s.size()!=t.size())
            return false;

        int freq[26]={0};

        for(char c:s)
            freq[c-'a']++;

        for(char c:t)
        {
            freq[c-'a']--;

            if(freq[c-'a']<0)
                return false;
        }

        return true;
    }
};
```

Remember

Increase with first string.
Decrease with second string.

TC: `O(n)` | **SC:** `O(1)`

8. Merge Sorted Array (88)

Pattern

Three Pointers (From End)

```
class Solution {
public:
   void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {

       int i=m-1;
       int j=n-1;
       int k=m+n-1;

       while(i>=0 && j>=0)
       {
          if(nums1[i]>nums2[j])
             nums1[k--]=nums1[i--];
          else
             nums1[k--]=nums2[j--];
       }

       while(j>=0)
          nums1[k--]=nums2[j--];
   }
};
```

Remember

Fill from the end because free space is at the end.

TC: `O(m+n)` | **SC:** `O(1)`

9. Valid Palindrome (125)

Pattern

Two Pointers (Opposite Ends)

```
class Solution {
public:
   bool isPalindrome(string s) {

       int left=0;
       int right=s.size()-1;

       while(left<right)
       {
          if(!isalnum(s[left]))
          {
             left++;
             continue;
          }

          if(!isalnum(s[right]))
          {
             right--;
             continue;
          }

          if(tolower(s[left])!=tolower(s[right]))
             return false;

          left++;
          right--;
       }

       return true;
   }
};
```

Remember

Skip invalid characters.
Compare lowercase letters. **TC:** `O(n)` | **SC:** `O(1)`

10. Move Zeroes (283)

Pattern

Two Pointers (Reader-Writer)

```
class Solution {

public:

    void moveZeroes(vector<int>& nums) {

        int k=0;

        for(int i=0;i<nums.size();i++)

        {

            if(nums[i]!=0)

            {

                nums[k]=nums[i];

                k++;

            }}

        while(k<nums.size())

        {

            nums[k]=0;

            k++;

        }

    }

};
```

Remember

First move all non-zero elements.
 Then fill remaining positions with zero.

TC: O(n) | **SC:** O(1)